

Document de fin d'itération 4

Table des matières

1	Gestion de projet	1
1.1	Rythme ralenti par les congés	1
1.2	Fin d'itération difficile	1
1.3	Mauvaise division des tâches	1
2	Qualité de code	2
2.1	Mauvais choix initiaux	2
2.1.1	Services stateful	2
2.1.2	Services trop interdépendants	2
2.1.3	Un seul type de bugin	2
2.1.4	Gestion des actions en combat	2
2.1.5	Création précoce de certains objets	3
2.2	Améliorations	3
2.2.1	Services	3
2.2.2	Interfaces pour les repositories	3
2.2.3	Différents types de bugin (composition)	3
2.2.4	Système d'actions en combat	4
2.2.5	Reporter la création des objets	4
2.3	Processus de refactoring	4

1 Gestion de projet

1.1 Rythme ralenti par les congés

Durant la semaine précédant les congés, le rythme a fortement ralenti, et n'a pas vraiment repris pendant les congés. Lors de la reprise, nous avons donc une quantité de travail supérieure à celle attendue.

1.2 Fin d'itération difficile

Quelques jours avant la deadline initiale, nous arrivions progressivement à la fin du projet, mais non sans difficultés. En effet, le code ressemblait de plus en plus à un « spaghetti géant » : il n'était pas bien pensé pour accueillir les nouvelles fonctionnalités de l'itération 4, et nous n'avions plus le temps d'entamer un refactoring au moment où nous en avons pris conscience.

C'est pourquoi, lorsque nous avons appris que la deadline était repoussée, nous avons immédiatement entamé un refactoring complet (cf. Chapitre 2).

Avec le recul, nous réalisons à présent que nous aurions vraisemblablement dû demander un refactoring dès l'itération 3. Effectuer ce dernier plus tôt aurait probablement permis de le rendre plus court que celui que nous avons dû réaliser pendant la période supplémentaire.

1.3 Mauvaise division des tâches

Nous avons divisé l'histoire 17 (Arbre de compétences) en les tâches suivantes :

- logique pour appliquer les effets
- controller skill tree
- model skill tree
- vue skill tree

La personne s'occupant de la vue n'a pas lu la partie de l'histoire indiquant que les positions des nœuds étaient déjà présentes dans le fichier `skill_tree.json` (ou du moins ne s'en est pas

souvenue), et a par conséquent réimplémenté inutilement un algorithme d'organisation des positions des nœuds. (Nous sommes revenus en arrière pour utiliser les coordonnées présentes dans le fichier, afin de respecter la demande du client.)

2 Qualité de code

Afin de rendre le refactoring le plus efficace possible, nous nous sommes fortement aidés des LLMs (Claude), majoritairement pour comprendre nos mauvais choix de conception et nos erreurs de compréhension de certains concepts tels que les services, ainsi que pour générer la Javadoc (sous notre supervision).

2.1 Mauvais choix initiaux

Dans cette section, nous décrivons brièvement les mauvais choix de conception que nous avons faits *avant le refactoring final*.

2.1.1 Services stateful

Quatre jours avant la remise, nous avons réalisé que nos services étaient beaucoup trop *stateful*. Par exemple, BugemonService stockait les LevelUps pas encore choisis, ce qui compliquait fortement son utilisation.

2.1.2 Services trop interdépendants

À de nombreuses reprises, des services étaient injectés dans d'autres services. Nous avons par la suite réalisé que cela était généralement une mauvaise idée, du fait du risque d'apparition de dépendances cycliques entre ceux-ci.

2.1.3 Un seul type de bugemon

Nous avions un seul type Bugemon, qui était utilisé partout. Celui-ci stockait toutes les informations d'un bugemon (incluant notamment les champs hp et niveau).

Les bugemons « par défaut » étaient stockés dans une liste au sein du BugemonService ; cette liste était obtenue grâce au repository que gérait ce service.

On utilisait ensuite le constructeur de copie de Bugemon pour créer les bugemons du joueur et de l'adversaire (à l'aide du cache décrit plus haut).

Nous avons réalisé que ce design présente plusieurs défauts. Notamment, le niveau dépend du joueur ; il est donc incorrect de le stocker dans des bugemons qui n'appartiennent à aucun joueur.

Lorsque le joueur lançait un combat *standalone* (hors du mode Tour), nous devions restaurer l'attribut hp des bugemons de son équipe.

2.1.4 Gestion des actions en combat

Afin de récupérer les actions effectuées en combat, nous utilisons un objet Trainer par joueur. Ceux-ci étaient conçus pour n'être utilisés que pendant un seul combat et possédaient notamment un champ stockant une action ainsi qu'un autre pour l'équipe ; ils étaient passés en paramètre à Combat.

Le controller de combat passait les Trainers au Combat. À chaque tour, il mettait à jour l'action stockée dans le Trainer correspondant au joueur, puis demandait au Combat d'effectuer le tour. Le Combat interrogeait alors les Trainers pour récupérer les actions qu'ils stockaient.

Ce système est fragile : la responsabilité du Trainer du joueur n'est pas clairement établie, car celui-ci est partagé entre le Combat et le controller.

2.1.5 Création précoce de certains objets

Nous avons à plusieurs reprises créé des objets plus tôt que nécessaire, ce qui rendait leur gestion inutilement complexe.

Exemple : dans le mode Tour NO, les Rooms pouvaient contenir des Combats. Nous créions ces combats directement à la création des Rooms.

Créer le Combat à la création de la Room était difficile, puisque cela nécessitait également de déjà créer un Trainer pour le joueur ainsi qu'un autre pour son adversaire.

2.2 Améliorations

2.2.1 Services

Nos services sont à présent pratiquement tous stateless.

Nous avons limité autant que possible le nombre de services injectés dans d'autres services.

Notre SaveService joue le rôle de *Facade* pour la sauvegarde du jeu, et prend donc en paramètre d'autres services.

2.2.2 Interfaces pour les repositories

Nous avons ajouté des interfaces à nos repositories. Cela a permis de grandement simplifier le code de ces derniers, en limitant le nombre de méthodes à implémenter.

2.2.3 Différents types de bagemons (composition)

Nous utilisons à présent différents types de Bugemons, en tirant parti de la composition. Ce design est inspiré du schéma de la base de données. Cela nous évite de devoir restaurer la vie des bugemons à la fin des combats standalone, puisqu'il nous suffit de recréer des RunBugemons (la run étant terminée après un combat standalone).

Bugemon
id
name
hp (default hp)
attack (default attack)
defense (default defense)
initiative (default initiative)
type
attacks (default attacks)
spritePath
isStarter
isBoss

Tableau 1. – Bugemon « atomique »

PlayerBugemon
Bugemon
level
xp
bonusHp
bonusAttack
bonusDefense
bonusInitiative
attacks

Tableau 2. – Bugemon d'un joueur

RunBugemon
Bugemon
currentHp

Tableau 3. – Bugemon d’une run

CombatBugemon
RunBugemon
participated
currentHp
effets

Tableau 4. – Bugemon durant un combat

2.2.4 Système d’actions en combat

Pour les combats, nous utilisons le *Strategy pattern*, avec le type `CombatStrategy`.

```
public interface CombatStrategy {
    void chooseAction(CombatContext ctx, ActionCallback callback);

    void chooseSwitch(CombatContext ctx, ActionCallback callback);
}
```

Grâce à ce pattern, nous n’avons plus besoin de l’objet `Trainer`. Du point de vue du combat, il y a simplement deux stratégies auxquelles il peut demander de choisir une action, ou bien imposer un switch (ce qui se produit lorsque le bugemon courant est KO).

Lorsqu’il demande une action (ou un switch), il passe deux arguments :

- `CombatContext` : un contexte contenant les informations du joueur et de son adversaire
- `ActionCallback` : le callback que la stratégie doit appeler pour notifier le combat de l’action prise par celle-ci

2.2.5 Reporter la création des objets

Notre refactoring tente de repousser la création des objets le plus tard possible (pour contrer les problèmes décrits dans Chapitre 2.1.5).

La `Room` contient simplement un `RoomType`, au lieu de stocker les combats dans les `CombatRooms`. Le combat est créé lorsque le joueur visite la `Room`, si le `RoomType` indique qu’il s’agit d’une room de combat.

2.3 Processus de refactoring

1. création de la branche `global-refactor`
2. suppression de la quasi-totalité du code existant
3. focalisation sur notre composition de bugemons, afin de s’assurer que ce modèle était bon (cf. Chapitre 2.2.3)
4. une fois satisfaits de notre composition de bugemons, nous avons attaqué en parallèle :
 - la vue de la gestion de l’équipe, celle-ci ne nécessitant que peu de modèle
 - la logique de combat
5. controller et vue de combat
6. repositories (ajout des interfaces notamment)
7. services
8. controllers et vues (plus ou moins en parallèle)

Durant ce processus, nous réintégrons essentiellement du code provenant de la branche `development`, en l’adaptant à la nouvelle architecture.